Refatoração de Testes e Detecção de Test Smells

Disciplina: Teste de Software - PUC Minas (Prof. Cleiton Tavares)
Aluno: Rafael de Oliveira Caldeira Lopes **Matrícula:** 829981
Ferramentas: Jest e ESLint (eslint-plugin-jest)
Repositório: https://github.com/RafaelLopes1810/test-smelly.git

1. Análise de smells

O projeto é um serviço simples de usuários (`src/userService.js`) com uma suíte propositalmente ruim em `test/userService.smelly.test.js`. Ela tem 5 testes: 4 passam e 1 está ignorado. Ou seja, tudo "verde", mas com vários maus cheiros. Li o arquivo antes de rodar qualquer ferramenta e destaco três smells, explicando por que são problemáticos e qual o risco de cada um.

Smell 1: Conditional Logic (lógica condicional)

```
for (const user of todosOsUsuarios) {
 const resultado = userService.deactivateUser(user.id);
 if (!user.isAdmin) {
 expect(resultado).toBe(true);
 const usuarioAtualizado = userService.getUserById(user.id);
 expect(usuarioAtualizado.status).toBe('inativo');
 } else {
 expect(resultado).toBe(false);
 }
}
```

Teste não deveria ter `for` nem `if`: ele precisa ser linear e determinístico. Aqui um único teste mistura dois cenários (usuário comum e administrador) e decide no meio do caminho quais verificações rodar. O risco é um cenário não executar o `expect` que deveria e o teste passar sem provar nada. Além disso, se falhar, não fica claro qual regra de negócio quebrou. O mesmo problema aparece no `try/catch` do teste de menoridade.

Smell 2: The Giant / Eager Test (teste ansioso)

```
test('deve criar e buscar um usuário corretamente', () => {
 const usuarioCriado = userService.createUser(
 dadosUsuarioPadrao.nome,
 dadosUsuarioPadrao.email,
 dadosUsuarioPadrao.idade
 );
 expect(usuarioCriado.id).toBeDefined();

 const usuarioBuscado = userService.getUserById(usuarioCriado.id);
 expect(usuarioBuscado.nome).toBe(dadosUsuarioPadrao.nome);
 expect(usuarioBuscado.status).toBe('ativo');
});
```

O próprio título entrega: o teste verifica criar e buscar, dois comportamentos (dois métodos) de uma vez. É um mau cheiro porque, quando ele quebra, não dá para saber se o defeito está na criação ou na busca, e uma mudança em qualquer um dos dois obriga a mexer no mesmo teste. O risco é perder tempo de diagnóstico e ter uma suíte difícil de manter.

Smell 3: teste que passa sem verificar nada (falha silenciosa em exceção)

```
try {
 userService.createUser('Menor', 'menor@email.com', 17);
} catch (e) {
 expect(e.message).toBe('O usuário deve ser maior de idade.');
}
```

Parente próximo da lógica condicional: nenhuma linha afirma que a chamada deve lançar uma exceção. Se alguém remover a validação de idade, `createUser` retorna normalmente, o `catch` nunca executa, nenhum `expect` roda e o teste continua passando. O risco é um falso positivo que esconde uma regressão justamente na regra que o teste deveria proteger.

Outros smells que encontrei: Fragile Test (o teste do relatório exige uma linha inteira montada com ID, pontuação e quebra de linha exatos, então qualquer ajuste de formatação o quebra); **General Fixture** leve (o

objeto `dadosUsuarioPadrao` fica no escopo global do arquivo, mas só um teste usa); **Assertion Roulette** localizado (várias expectativas de comportamentos diferentes no mesmo teste); e um **teste desabilitado** (`test.skip`, sem nenhuma asserção) para o relatório com lista vazia. Não encontrei Mystery Guest, Sleeper nem Redundant Assertion.

2. Relatório da ferramenta (ESLint)

Instalei `eslint` e `eslint-plugin-jest` e criei o `.eslintrc.json` do enunciado. A instalação trouxe o ESLint 10, que não usa mais o `.eslintrc.json` como padrão, então troquei para o **ESLint 8.57.1** (com `eslint-plugin-jest` 29.17.0). Rodei `npx eslint .` antes de refatorar e o resultado foi de **6 problemas: 4 erros e 2 avisos**, todos no arquivo smelly:

Regra do ESLint	Ocorrências	Linhas	Nível
`jest/no-conditional-expect`	4	44, 46, 49 e 73	Erro
`jest/no-disabled-tests`	1	77	Aviso
`jest/expect-expect`	1	77	Aviso

```
PS C:\Users\T-GAMER\Documents\GitHub\test-smelly> npx eslint test/userService.smelly.test.js

C:\Users\T-GAMER\Documents\GitHub\test-smelly\test\userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped            jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
```

Figura 1 - Primeira execução do ESLint no arquivo smelly.

A ferramenta automatizou bem a parte "mecânica": apontou, em segundos e com linha exata, os `expect` dentro de ramos condicionais (os do `if/else` e o do `catch`) e o teste ignorado e sem asserções. Isso confirma parte do Smell 1 e o teste desabilitado. Por outro lado, ela **não** viu o resto: não classificou o teste que junta criar e buscar como Eager Test, não percebeu a fragilidade do relatório, não reclamou da fixture global e não notou que o `try/catch` passa sem exceção. A regra `no-conditional-expect` só olha onde o `expect` está, não o que o teste realmente garante. Por isso a análise manual continua necessária: o linter pega o que tem regra, e smells de design pedem olho humano.

3. Processo de refatoração

Criei `test/userService.clean.test.js` como cópia do original e refatorei só ele (o smelly e o `src/` ficaram intactos). Obs.: no repositório a pasta se chama `test/` e não `__tests__/`, então mantive a estrutura real. Escolhi o teste de desativação por juntar mais problemas.

Antes (userService.smelly.test.js)

```
test('deve desativar usuários se eles não forem administradores', () => {
 const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
 const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40,

true);  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

 for (const user of todosOsUsuarios) {
 const resultado = userService.deactivateUser(user.id);
 if (!user.isAdmin) {
 expect(resultado).toBe(true);
 const usuarioAtualizado = userService.getUserById(user.id);
 expect(usuarioAtualizado.status).toBe('inativo');
 } else {
 expect(resultado).toBe(false);
 }
 }
});
```

Depois (userService.clean.test.js)

```
test('deve desativar usuário comum e atualizar seu status', () => {
 // Arrange
 const userService = criarServicoLimpo();
 const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

 // Act
 const resultado = userService.deactivateUser(usuarioComum.id);

 // Assert
 expect(resultado).toBe(true);
 expect(userService.getUserById(usuarioComum.id).status).toBe('inativo');
});

test('não deve desativar usuário administrador', () => {
 // Arrange
 const userService = criarServicoLimpo();
 const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

 // Act
 const resultado = userService.deactivateUser(usuarioAdmin.id);

 // Assert
 expect(resultado).toBe(false);
 expect(userService.getUserById(usuarioAdmin.id).status).toBe('ativo');
});
```

Decisões que tomei: (1) **Separei os dois cenários em dois testes**, o que eliminou o `for` e o `if/else` (Conditional Logic) e também o teste "faz-tudo" (Eager Test): agora cada teste tem um motivo para falhar. (2) **Apliquei o AAA** com comentários e linhas em branco, então dá para ler cada fase sem esforço. (3) **Nomes descritivos**, que dizem o comportamento esperado e viram documentação. (4) No cenário do administrador, **acrescentei a checagem de que o status continua "ativo"**; no original só se olhava o retorno `false`. Nada do que era verificado foi perdido.

O mesmo vale para o resto do arquivo: criação e busca viraram testes separados (com `toMatchObject` para agrupar as propriedades); a menoridade usa `expect(fn).toThrow(...)`, que falha se a exceção não vier (corrige a falha silenciosa); o relatório passou a usar `toContain` em vez de comparar uma linha exata (Fragile Test); os dados saíram da fixture global e agora ficam no Arrange de cada teste, com o helper `criarServicoLimpo()` no lugar do `beforeEach` (General Fixture); e o teste do relatório vazio foi habilitado e ganhou uma asserção de verdade.

Validação final	**Resultado**
`npx eslint test/userService.clean.test.js`	Nenhum erro e nenhum aviso
`npx eslint .`	Só os 6 problemas já conhecidos, todos no arquivo smelly (o clean não aparece)
`npm test`	2 suítes passaram: clean com 7 testes; smelly com 4 passando e 1 ignorado. Total: 11 aprovados, 1 ignorado
Arquivos protegidos	`src/userService.js` e o arquivo smelly sem nenhuma alteração

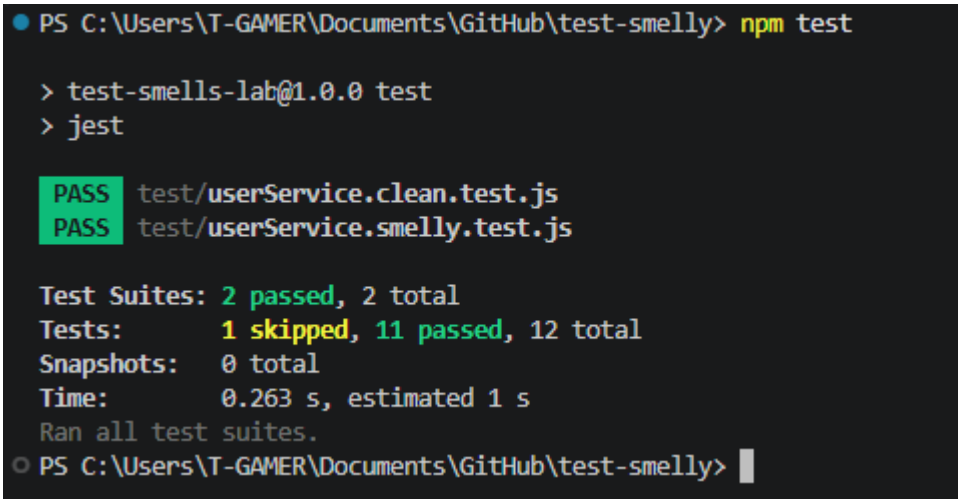


Figura 2 - Execução completa da suíte após a refatoração.

4. Conclusão

Teste verde não é sinônimo de teste bom. A suíte original passava, mas tinha testes que podiam passar sem verificar nada, que misturavam vários comportamentos e que quebrariam por qualquer mudança de formatação. Ao refatorar com AAA, um cenário por teste e asserções que realmente falham quando o comportamento muda, os testes ficaram mais fáceis de ler, de diagnosticar e de manter, e mais capazes de achar bugs.

A análise estática entra como uma primeira rede de segurança: barata, rápida e automática, pode rodar a cada commit ou no CI para impedir que certos maus cheiros voltem. Mas ela só pega o que tem regra e não substitui a revisão humana. Tratar testes como código de primeira classe é o que mantém a suíte confiável conforme o projeto cresce.